Further Optimizations and Multi-Input Einsums

Max Koch

Today's Agenda

- Recap: L2 optimization via operations on contraction configuration
- Configurations with multiple primitive dimensions per type
- Multi-input einsums

Recap: L2 Optimization via Operations on Contraction Configuration

- Since the configuration's execution order is defined from right to left, we can achieve L2 reuse by optimizing the configuration (equivalent to swizzling inside the kernel):

1. Fuse and split dimensions to retrieve dimensions of a specific target size for L2 reuse
2. Move those dimensions to the right inside the configuration (through permutation)
3. Use the other dimensions (on the left) for parallelization or looping inside the kernel

- General pattern for L2 reuse optimization:

```
# general L2 reuse pattern
# ========================

# m_l2, n_l2 used for L2 reuse,
# prim_m, prim_n, prim_k used for the GEMM primitive
# [...] - all other dimensions

config.dim_sizes = [ [...], |m_l2|, |n_l2|, |k_seq|, |prim_m|, |prim_n|, |prim_k| ]
```

Recap: L2 Optimization via Operations on Contraction Configuration

- Live Session 🤦‍♂️

Recap: L2 Optimization via Operations on Contraction Configuration

- In the general case, the shown dimensions are not necessarily a single dimension, but can be a set of dimensions
  - `m_l2` and `n_l2` are sets of dimensions used for L2 cache reuse
  - `k_seq` is a set of dimensions containing all K dimensions not used for the primitive
  - `prim_m`, `prim_n`, `prim_k` are the dimensions used for the primitive (e.g., matmul)

```
# general L2 reuse pattern
# ========================

# m_l2, n_l2 used for L2 reuse,
# prim_m, prim_n, prim_k used for the GEMM primitive
# [...] - all other dimensions

config.dim_sizes = [ [...], |m_l2|, |n_l2|, |k_seq|, |prim_m|, |prim_n|, |prim_k| ]
```

Multi-Primitive Contraction Kernels

- So far, we have only considered configurations containing a single primitive dimension per type (M, N, K)

```
@ct.kernel
def contraction(A, B, C):
  pid = ct.bid(0)

  # Decompose pid -> retrieve dimension indices for parallel dimensions
  par_0 = ...
  par_1 = ...

  # Loop over all sequential dimensions (including contraction dimensions)
  for it_seq_0 in size_K:
    # Compute tile of C using a (B)GEMM
    tA = ct.load(A, (par_0, it_seq_0, 0, 0), (1, 1, |prim_m|, |prim_k|))
    tB = ct.load(B, (it_seq_0, par_1, 0, 0), (1, 1, |prim_k|, |prim_n|))

    # reshape and call ct.mma

  # Reshape output tile
  # Store tile of C to global memory
```

Multi-Primitive Contraction Kernels

- Multiple primitive dimensions per K-type

```
@ct.kernel
def contraction(A, B, C):
  pid = ct.bid(0)

  # Decompose pid -> retrieve dimension indices for parallel dimensions
  par_0 = ...
  par_1 = ...

  # Compute tile of C using a (B)GEMM
  tA = ct.load(A, (par_0, 0, 0, 0), (1, |prim_k_1|, |prim_m|, |prim_k_0|))
  tB = ct.load(B, (0, par_1, 0, 0), (|prim_k_1|, 1, |prim_k_0|, |prim_n|))
  # permute tiles (local TTGT)
  # reshape and call ct.mma

  # Reshape output tile
  # Store tile of C to global memory
```

Multi-Primitive Contraction Kernels

- Multiple primitive dimensions per M-type

```
@ct.kernel
def contraction(A, B, C):
  pid = ct.bid(0)

  # Decompose pid -> retrieve dimension indices for parallel dimensions
  par_0 = ...
  par_1 = ...

  # Loop over all sequential dimensions (including contraction dimensions)
  for it_seq_0 in size_K:
    # Compute tile of C using a (B)GEMM
    ct.load(A, (0, par_1, it_seq_0, 0, 0), (|prim_m_1|, 1, 1, |prim_m_0|, |prim_k|))
    ct.load(B, (it_seq_0, par_1, par_0, 0, 0), (1, 1, 1, |prim_k|, |prim_n|))

    # reshape and call ct.mma

  # Reshape output tile
  # Store tile of C to global memory
```

Multi-Primitive Contraction Kernels

- We can express multi-primitive kernels in the configuration by allowing multiple primitive dimensions per type

```
config.dim_types =
  [ config.m,    config.n,   config.k,    config.m,    config.n,    config.k ]

config.exec_types =
  # par_0,       par_1,      it_seq_0,    prim_m,      prim_n,      prim_k
  [ config.par,  config.par, config.seq,  config.prim, config.prim, config.prim ]

config.exec_types_multiprim_k =
  # par_0,       par_1,      prim_k_1,    prim_m,      prim_n,      prim_k_0
  [ config.par,  config.par, config.prim, config.prim, config.prim, config.prim ]

config.exec_types_multiprim_m =
  # prim_m_1,    par_1,      it_seq_0,    prim_m_0,    prim_k,      prim_n
  [ config.prim, config.par, config.seq,  config.prim, config.prim, config.prim ]

```

Loading Tiles

- As of now, we have only considered loading tiles inside the innermost loop of the kernel

```
 1 @ct.kernel
 2 def contraction(A, B, C):
 3   pid = ct.bid(0)
 4   # [...]
 5
 6   for m_seq in size_m_seq:
 7     for k_seq in size_k_seq:
 8       tA = ct.load(A, (..., 0, 0), (..., |prim_m|, |prim_k|))
 9       tB = ct.load(B, (..., 0, 0), (..., |prim_k|, |prim_n|))
10       # [...]
11
12   # Reshape output tile
13   # Store tile of C to global memory
```

Loading Tiles

- It can be beneficial to pre-load multiple tiles
  - For example, it is beneficial if

```
@ct.kernel
def contraction(A, B, C):
  pid = ct.bid(0)
  # [...]

  tA = ct.load(A, (..., 0, 0, 0, 0), (..., |m_seq|, |k_seq|, |prim_m|, |prim_k|))

  for m_seq in size_m_seq: # m_seq has stride 1
    for k_seq in size_k_seq:
      ct.extract(tA, (..., m_seq, k_seq, 0, 0), (..., 1, 1, |prim_m|, |prim_k|))
      ct.load(B, (..., 0, 0), (..., |prim_k|, |prim_n|))
      # [...]

  # Reshape output tile
  # Store tile of C to global memory
```

Loading Tiles

- It can be beneficial to pre-load multiple tiles
  - For example, it is beneficial if the primitive dimension does not have stride 1

```
@ct.kernel
def contraction(A, B, C):
  pid = ct.bid(0)
  # [...]

  tA = ct.load(A, (..., 0, 0, 0, 0), (..., |m_seq|, |k_seq|, |prim_m|, |prim_k|))

  for m_seq in size_m_seq: # m_seq has stride 1
    for k_seq in size_k_seq:
      ct.extract(tA, (..., m_seq, k_seq, 0, 0), (..., 1, 1, |prim_m|, |prim_k|))
      ct.load(B, (..., 0, 0), (..., |prim_k|, |prim_n|))
      # [...]

  # Reshape output tile
  # Store tile of C to global memory
```

Multi-Input Einsums

- So far, we have only considered einsums with two input tensors (e.g., matmul: `mk, kn -> mn`)
- Let's consider a matrix chain multiplication `E = A @ B @ C @ D`
- Einsum:

It is possible to compute the contraction in one step:

Multi-Input Einsums

- So far, we have only considered einsums with two input tensors (e.g., matmul: `mk, kn -> mn`)
- Let's consider a matrix chain multiplication `E = A @ B @ C @ D`
- Einsum: `ab, bc, cd, de -> ae`

It is possible to compute the contraction in one step:

Multi-Input Einsums

- So far, we have only considered einsums with two input tensors (e.g., matmul: `mk, kn -> mn`)
- Let's consider a matrix chain multiplication `E = A @ B @ C @ D`
- Einsum: `ab, bc, cd, de -> ae`

It is possible to compute the contraction in one step:

$$E[a, e] = \sum_{b,c,d} A[a, b] \times B[b, c] \times C[c, d] \times D[d, e]$$

- FLOPs: $4 * |a| * |b| * |c| * |d| * |e|$

Multi-Input Einsums

- So far, we have only considered einsums with two input tensors (e.g., matmul: `mk, kn -> mn`)
- Let's consider a matrix chain multiplication `E = A @ B @ C @ D`
- Einsum: `ab, bc, cd, de -> ae`

It is possible to compute the contraction in one step:

$$E[a, e] = \sum_{b,c,d} A[a, b] \times B[b, c] \times C[c, d] \times D[d, e]$$

- FLOPs: $4 * |a| * |b| * |c| * |d| * |e|$

It is beneficial for the FLOP count to compute the contraction in multiple binary steps:

1. `ab, bc -> ac` 2. `ac, cd -> ad` 3. `ad, de -> ae`

- FLOPs: $2 * (|a| * |b| * |c| + |a| * |c| * |d| + |a| * |d| * |e|)$

Multi-Input Einsums

It is beneficial for the FLOP count to compute the contraction in multiple binary steps:

1. `ab, bc -> ac` 2. `ac, cd -> ad` 3. `ad, de -> ae`

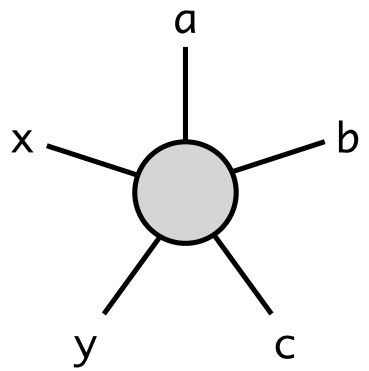
- FLOPs: $2 * (|a| * |b| * |c| + |a| * |c| * |d| + |a| * |d| * |e|)$

- Looking at multi-input einsums, we further increase the size of our *optimization space*:
  - **Contraction sequence** defines the order of binary contractions
    - Has an impact on FLOP count and size of intermediate tensors
    - Tools like `opt_einsum` can be used to find a good contraction sequence
  - Memory layouts of the **intermediate tensors** can be chosen freely (e.g., `ac`, `ad`)
    - Choose layouts to optimize for memory access patterns

Light-Field Dataset (https://lightfields.stanford.edu/mvlf/)

- 5D tensor representing multiple images along two axes for camera position

ID	Description	Size
a	Vertical camera index	4
b	Horizontal camera index	4
c	Color channel	3
y	Image height	1536
x	Image width	1280

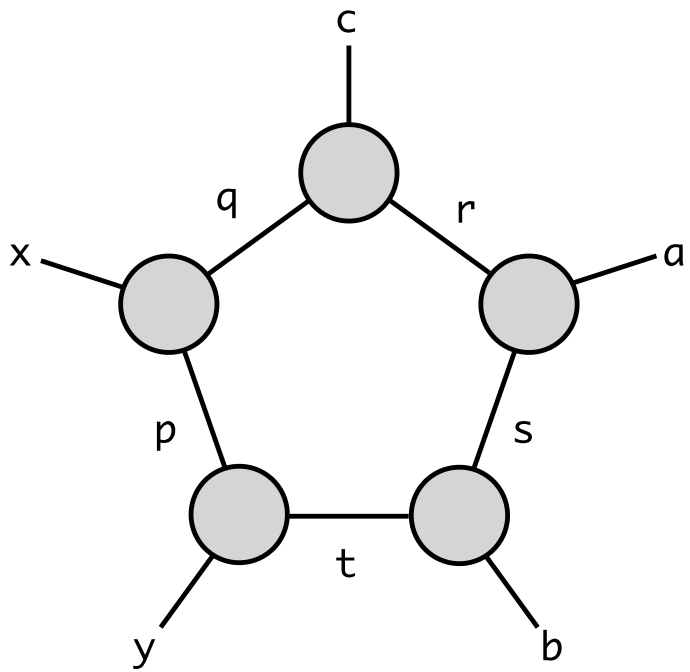


$$|a| \cdot |b| \cdot |c| \cdot |y| \cdot |x|$$
$$= 4 \cdot 4 \cdot 3 \cdot 1536 \cdot 1280$$
$$94{,}371{,}840$$

Light-Field Dataset (https://lightfields.stanford.edu/mvlf/)

- Tensor ring decomposition

ID	Description	Size
a	Vertical camera index	4
b	Horizontal camera index	4
c	Color channel	3
y	Image height	1536
x	Image width	1280
p, q, r, s, t	Ranks used in the tensor ring decomposition	64



$$
\begin{aligned}
&|r| \cdot |a| \cdot |s| + |s| \cdot |b| \cdot |t| \\
&+ |t| \cdot |y| \cdot |p| + |p| \cdot |x| \cdot |q| \\
&= 64^2 \cdot \left(4 + 4 + 3 + 1536 + 1280\right) \\
&= 11{,}579{,}392
\end{aligned}
$$